

GENETIC-ALGO-WEEK1

```
import random

p = input("Enter initial population: ").split()
n = len(p)
pc = 0.8
pm = 0.01
gen = 10

def fit(s):
    x = int(s, 2)
    return x, x * x

for g in range(gen):
    f = [fit(s)[1] for s in p]
    total = sum(f)

    if g == 0:
        print("\nString\tx\tFitness\tProbability\tExpected Count")
        for i in range(n):
            prob = f[i] / total
            print(p[i], "\t", int(p[i], 2), "\t", f[i],
                  "\t", round(prob * 100, 2), "%",
                  "\t", round(prob * n, 2))

    parents = random.choices(p, weights=f, k=n)
    new = []

    for i in range(0, n, 2):
        a, b = parents[i], parents[(i + 1) % n]

        if random.random() < pc:
            k = random.randint(1, len(a) - 1)
            a, b = a[:k] + b[k:], b[:k] + a[k:]

        a = ''.join(
            '1' if c == '0' and random.random() < pm
            else '0' if c == '1' and random.random() < pm
            else c for c in a
        )

        b = ''.join(
            '1' if c == '0' and random.random() < pm
            else '0' if c == '1' and random.random() < pm
            else c for c in b
        )
```

```

        new += [a, b]

    p = new[:n]

best = max(p, key=lambda s: fit(s)[1])

print("\nBest String:", best)
print("x =", int(best, 2))
print("Maximum Fitness =", fit(best)[1])

```

OUTPUT

```

... Enter initial population: 10111 00101 01100 01110

String  x      Fitness Probability      Expected Count
10111   23      529      59.17 %      2.37
00101    5       25       2.8 %    0.11
01100   12      144      16.11 %      0.64
01110   14      196      21.92 %      0.88

Best String: 11111
x = 31
Maximum Fitness = 961

```
